**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**
**НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”**

**Кафедра систем штучного інтелекту**

# Лабораторна робота №2

з дисципліни
«Операційні системи»

**Виконав:**
студент групи КН-114
Добрій Назарій

**Викладач:**
Кривенчук Ю.П.

Львів – 2020 р.

**Тема**. Робота з процесами в ОС Windows.

**Мета**. Ознайомитися з багатопоточністю в ОС Windows. Навчитися працювати з процесами, використовуючи WinAPI-функції.

**Завдання.**

1. Створити окремий процес, і здійснити в ньому табулювання функції, задану розкладом в ряд Тейлора, в області її визначення на відрізку від A до B (кількість кроків не менше 100 000). Функцію взяти з у відповідності до номера функції та порядкового номера у журнальному списку.
2. Реалізувати табулювання функцій у 2-ох, 4-ох, 8-ох процесах. Виміряти час роботи процесів за допомогою функцій WinAPI. Порівняти результати роботи в одному і в багатьох процесах.
3. Для кожного процесу реалізувати можливість його запуску, зупинення, завершення та примусове завершення («вбиття»).
4. Реалізувати можливість зміни пріоритету виконання процесу.
5. Продемонструвати результати виконання роботи, а також кількість створених процесів у "Диспетчері задач", або подібних утилітах (н-д, ProcessExplorer)/

## Варіант 6

Список функцій для табулювання:

$$6.\quad \ln\left(\frac{1+x}{1-x}\right) = 2\sum_{k=1}^{\infty}\frac{1}{2k+1}x^{2k-1} = 2\left[\frac{x}{1}+\frac{x^3}{3}+\frac{x^5}{5}+\frac{x^7}{7}+\ldots+\frac{x^{2n+1}}{2n+1}+\ldots\right];\quad |x|<1$$

Скріни коду:

Табулювання функції:

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <string>
#include <iomanip>
#include <windows.h>
#include <conio.h>

using namespace std;

int main(int argv, char* argc[]) {
    double Left = atof(argc[0]), Right = atof(argc[1]), step = 0.00002;

    for (double x = Left; x < Right; x += step) {
        cout << "x = " << setprecision(6) << fixed << x << " \tres = " << pow((1 - x), -2) << endl;
    }

    _getch();
}
```

Головна програма:

```cpp
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <string>
#include <iomanip>
#include <windows.h>
#include <conio.h>
#include <stdio.h>
#include <set>

using namespace std;

double filetime_to_double(struct _FILETIME& ft) {
    unsigned long long t = ft.dwLowDateTime + ((unsigned long long)ft.dwHighDateTime << 32);
    return t / 1e7;
}

void kill(DWORD process_id) {
    HANDLE hnd;
    hnd = OpenProcess(SYNCHRONIZE | PROCESS_TERMINATE, TRUE, process_id);
    TerminateProcess(hnd, 0);
}

int main(int argv, int* arg) {
    STARTUPINFO si[9];
    PROCESS_INFORMATION pi[9];
    ZeroMemory(&si, sizeof(si));
    ZeroMemory(&pi, sizeof(pi));
    BOOL bCreateProcess[9];

    int features;
    cout << "Enter the features 1 - measure time, 2 - show processes: ";
    cin >> features;

    cout << "Please, enter the number of processes 1, 2, 4, 8: ";
    double Left, Right;
    double len, n;
    cin >> n;
    double divider = 100000 / n;
    string s;

    for (int i = 1; i <= n; i++) {
        Left = ((divider * i) / 50000) - 1;
        Right = ((divider * (double)(i + 1)) / 50000) - 1;

        s = to_string(Left) + " " + to_string(Right);
        const char* cstr = s.c_str();
        wchar_t par[50];
        mbstowcs(par, cstr, strlen(cstr) + 1);

        bCreateProcess[i] = CreateProcess(
            L"C:\\Users\\User\\source\\repos\\tabulation\\Debug\\tabulation.exe",
            par,
            NULL,
            NULL,
            FALSE,
            CREATE_NEW_CONSOLE,
            NULL,
            NULL,
            &si[i],
            &pi[i]
        );
    }

    if (features == 1)
    {
        for (int i = 1; i <= n; i++) {
            WaitForSingleObject(pi[i].hProcess, INFINITE);
            FILETIME creationTime, exitTime, kernelTime, userTime;
            GetProcessTimes(pi[i].hProcess, &creationTime, &exitTime, &kernelTime, &userTime);

            cout << "Time elapsed: " << filetime_to_double(exitTime) - filetime_to_double(creationTime) << " seconds" << endl;
        }
    }
```

```
else
{
    set <int> killed_collection;
    int cmd = -1;
    int priority;
    int num;

    do {
        cout << "Enter 1 to kill processes" << endl;
        cout << "Enter 2 to change priroity of process" << endl;
        cout << "Enter 3 to suspend process" << endl;
        cout << "Enter 4 to resume process: " << endl;
        cin >> cmd;

        if (cmd == 1)
        {
            cout << "Enter number of process to kill" << endl;
            cin >> num;
        }

        else if (cmd == 2)
        {
            cout << "Enter number of process to change priority" << endl;
            cin >> num;
        }

        else if (cmd == 3)
        {
            cout << "Enter number of process to syspend" << endl;
            cin >> num;
        }

        else if (cmd == 4)
        {
            cout << "Enter number of process to resume" << endl;
            cin >> num;
        }

        else
        {
            system("pause");
            return 0;
        }

        if (killed_collection.count(num) == 0) {
            if (cmd == 1) {
                kill(pi[num].dwProcessId);
                killed_collection.insert(num);
            }

            else if (cmd == 2) {
                cout << "Enter new priority: 1 - low, 2 - below normal, 3 - normal, 4 - above normal, 5 - high" << endl;
                cin >> priority;

                if (priority == 1) {
                    SetPriorityClass(pi[num].hProcess, IDLE_PRIORITY_CLASS);
                }

                else if (priority == 2) {
                    SetPriorityClass(pi[num].hProcess, BELOW_NORMAL_PRIORITY_CLASS);
                }

                else if (priority == 3) {
                    SetPriorityClass(pi[num].hProcess, NORMAL_PRIORITY_CLASS);
                }

                else if (priority == 4) {
                    SetPriorityClass(pi[num].hProcess, ABOVE_NORMAL_PRIORITY_CLASS);
                }

                else
                {
                    SetPriorityClass(pi[num].hProcess, HIGH_PRIORITY_CLASS);
                }
            }

            else if (cmd == 3)
            {
                SuspendThread(pi[num].hThread);
            }

            else if (cmd == 4)
            {
                ResumeThread(pi[num].hThread);
            }
        }

    } while (cmd != -1);
}

system("pause");
return 0;
}
```

Час виконання табуляції 2-ох процесів:

C:\Users\User\source\repos\tabulation\Debug\tabulation.exe

```
x = 1.021460    res = 2171.403656
x = 1.021480    res = 2167.361956
x = 1.021500    res = 2163.331531
x = 1.021520    res = 2159.312337
x = 1.021540    res = 2155.304333
x = 1.021560    res = 2151.307479
x = 1.021580    res = 2147.321732
x = 1.021600    res = 2143.347051
x = 1.021620    res = 2139.383395
x = 1.021640    res = 2135.430725
x = 1.021660    res = 2131.488999
x = 1.021680    res = 2127.558176
x = 1.021700    res = 2123.638217
x = 1.021720    res = 2119.729082
x = 1.021740    res = 2115.830730
x = 1.021760    res = 2111.943123
x = 1.021780    res = 2108.066220
x = 1.021800    res = 2104.199983
x = 1.021820    res = 2100.344372
x = 1.021840    res = 2096.499349
x = 1.021860    res = 2092.664875
x = 1.021880    res = 2088.840911
x = 1.021900    res = 2085.027418
x = 1.021920    res = 2081.224359
x = 1.021940    res = 2077.431696
x = 1.021960    res = 2073.649391
x = 1.021980    res = 2069.877405
x = 1.022000    res = 2066.115702
x = 1.022020    res = 2062.364245
x =
```

C:\Users\User\source\repos\tabulation\Debug\tabulation.exe

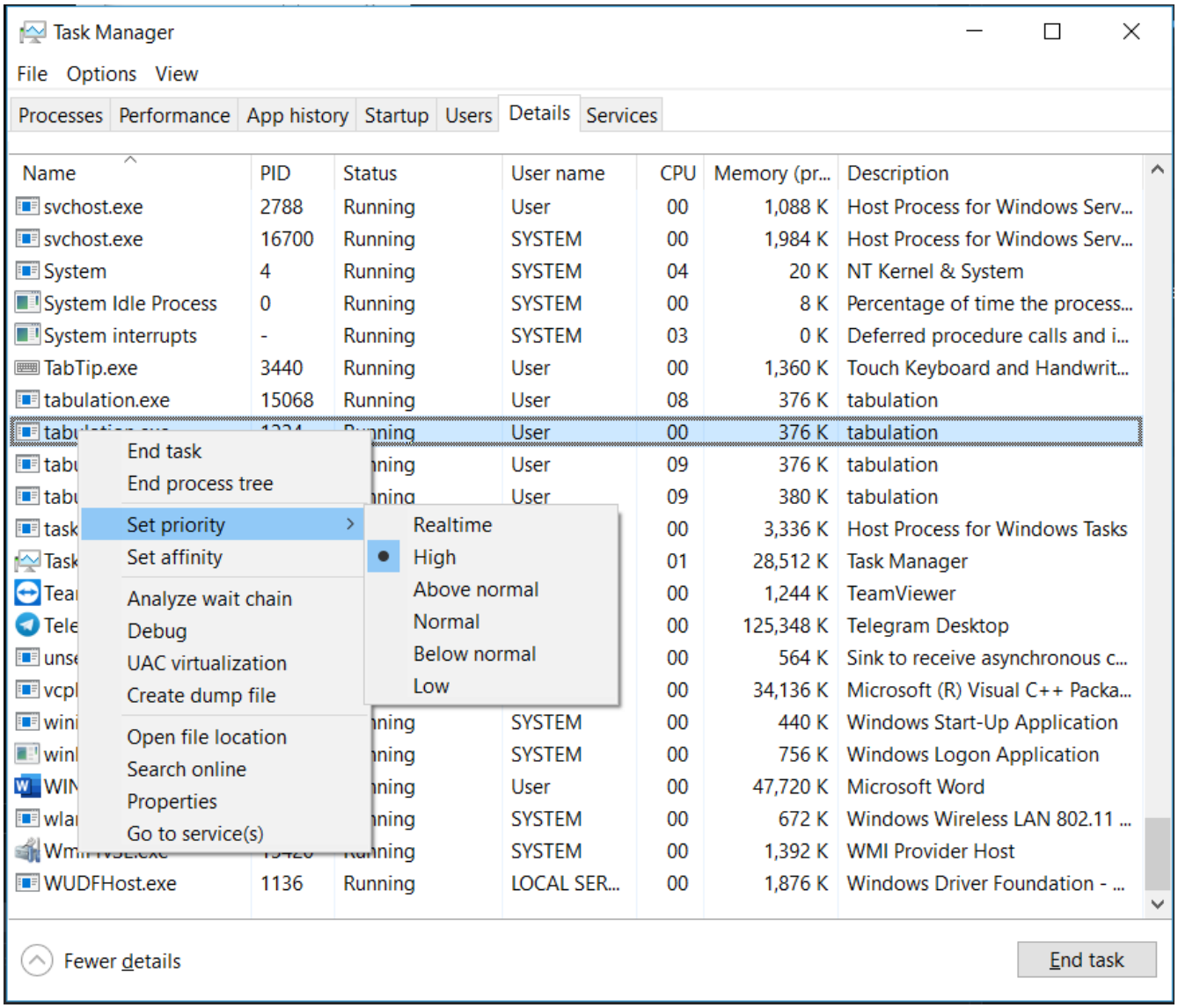
```
x = 0.115400    res = 1.277927
x = 0.115420    res = 1.277985
x = 0.115440    res = 1.278043
x = 0.115460    res = 1.278101
x = 0.115480    res = 1.278158
x = 0.115500    res = 1.278216
x = 0.115520    res = 1.278274
x = 0.115540    res = 1.278332
x = 0.115560    res = 1.278390
x = 0.115580    res = 1.278447
x = 0.115600    res = 1.278505
x = 0.115620    res = 1.278563
x = 0.115640    res = 1.278621
x = 0.115660    res = 1.278679
x = 0.115680    res = 1.278737
x = 0.115700    res = 1.278794
x = 0.115720    res = 1.278852
x = 0.115740    res = 1.278910
x = 0.115760    res = 1.278968
x = 0.115780    res = 1.279026
x = 0.115800    res = 1.279084
x = 0.115820    res = 1.279142
x = 0.115840    res = 1.279199
x = 0.115860    res = 1.279257
x = 0.115880    res = 1.279315
x = 0.115900    res = 1.279373
x = 0.115920    res = 1.279431
x = 0.115940    res = 1.279489
x = 0.115960    res = 1.279547
x = 0.115980    res = 1.27
```

```
C:\Users\User\source\repos\690\Debug\690.exe
Enter the features 1 - measure time, 2 - show processes: 1
Please, enter the number of processes 1, 2, 4, 8: 2
Time elapsed: 83.1479 seconds
Time elapsed: 85.2307 seconds
Press any key to continue . . .
```

Реалізація запуску, зупинення, завершення та примусове завершення («вбиття»):

```
C:\Users\User\source\repos\690\Debug\690.exe
Enter the features 1 - measure time, 2 - show processes: 2
Please, enter the number of processes 1, 2, 4, 8: 4
Enter 1 to kill processes
Enter 2 to change priroity of process
Enter 3 to suspend process
Enter 4 to resume process:
2
Enter number of process to change priority
2
Enter new priority: 1 - low, 2 - below normal, 3 - normal, 4 - above normal, 5 - high
5
Enter 1 to kill processes
Enter 2 to change priroity of process
Enter 3 to suspend process
Enter 4 to resume process:
```

Зміни процесів у диспечері задач:




Висновок: я ознайомився з багатопоточністю в ОС Windows. Навчитися працювати з процесами, використовуючи WinAPI-функції.